Practical:10	Implement Kruskal's algorithm.
Date : 19/10/2026	

Aim: To implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.

Program:

```
# Kruskal's Algorithm

vertices = 5

edges = [
    (2, 3, 4),
    (0, 1, 2),
    (1, 2, 3),
    (0, 3, 6),
    (1, 3, 8),
    (1, 4, 5),
    (2, 4, 7),
    (3, 4, 9)
]

# Sort edges by weight
edges.sort(key=lambda edge: edge[2])

parent = list(range(vertices))

# Find the parent of a vertex
def find(vertex):
    if parent[vertex] != vertex:
        parent[vertex] = find(parent[vertex])
    return parent[vertex]

# Join two sets
def union(u, v):
    parent[find(u)] = find(v)
```

```
total_cost = 0
selected_edges = 0

print("Edges in Minimum Spanning Tree:")

for u, v, weight in edges:
    if find(u) != find(v):
        union(u, v)

        print(u, "-", v, ":", weight)

        total_cost += weight
        selected_edges += 1

        if selected_edges == vertices - 1:
            break

print("Total cost of MST:", total_cost)
```

Output:-

```
C:\Users\Vishnu\OneDrive>python kruskals.py
Edges in Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 3
2 - 3 : 4
1 - 4 : 5
Total cost of MST: 14
```

Analysis:

Time Complexity: O(E log E)
Space Complexity: O(V)
Where:

- V = number of vertices
- E = number of edges

Sorting the edges takes O(E log E) time. The pare

Conclusion

Kruskal's Algorithm was implemented successfully using the greedy approach to find the Minimum Spanning Tree with a minimum total cost of 14.